CS/ECE/ISyE 524 — Introduction to Optimization — Spring 2026

Nutrition Optimizer

Ian Guo (yguo388@wisc.edu), Muxin Li (mli936@wisc.edu), Leopold Dong (ldong63@wisc.edu)

Table of Contents

1. Introduction

What does it cost, in dollars and in carbon, to eat well? This project uses multi-objective linear programming to design optimal daily diets that simultaneously minimize financial cost and greenhouse gas (GHG) emissions. By applying the epsilon-constraint method, we construct a Pareto frontier that makes the precise trade-off between eating economically and eating sustainably visible and quantifiable. We further examine how medically motivated dietary restrictions, specifically for diabetes and hypertension, shift this frontier and change the marginal cost of reducing carbon emissions.

The idea of optimizing a diet mathematically is not new. In 1939, economist George Stigler posed what became a landmark problem in operations research: what is the cheapest possible diet that still meets a person's basic nutritional needs? His answer, a monotonous regimen of wheat flour, cabbage, and pork liver, was solved by hand and later became one of the first problems solved by the simplex method when it was introduced by George Dantzig in 1947 (Dantzig, 1990). The "Stigler Diet" demonstrated that linear programming could find globally optimal solutions to real-world resource allocation problems, and diet optimization has served as a teaching example in optimization ever since.

Today, the challenge is considerably richer. The global food system accounts for roughly 26% of all greenhouse gas emissions worldwide (Poore & Nemecek, 2018), and the carbon footprint of what we eat varies enormously by food type. Beef, for instance, produces more than 60 kg of CO_2 equivalent per kilogram of food, while legumes produce less than 1 kg. As climate concerns grow alongside rising food costs, the ability to quantify and navigate the tension between affordability and

sustainability has become a genuinely practical policy question, not just an academic exercise.

Data Collection. This project integrates two publicly available, real-world datasets. Nutritional information comes from the CORGIS Food dataset, which provides macronutrient profiles, calories, protein, fat, carbohydrates, fiber, sodium, and saturated fat, for approximately 300 food items. Emissions data comes from Our World in Data, drawing on Poore & Nemecek's (2018) global meta-analysis of food lifecycle assessments, and provides kilograms of CO_2 equivalent per kilogram of food across roughly 40 broad food categories. The two datasets are merged by mapping each food item's category label (e.g., "Beef," "Poultry," "Milk") to its corresponding emissions entry.

Key Simplifying Assumptions. Because the two datasets do not share a common price column, food prices are synthetically generated using a repeating pattern of representative costs based on typical grocery store values. Emissions are assigned at the category level rather than the individual food item level, meaning all beef products share the same GHG intensity regardless of cut or preparation. Each food item is treated as a continuous decision variable representing daily servings, bounded between 0 and 5. Nutritional constraints are modeled as daily minimum (or maximum) thresholds based on standard dietary reference values, without accounting for meal structure, palatability, or variety.

Outline. The remainder of this report is organized as follows. Section 2 presents the mathematical model, defining the decision variables, objective function, constraints, and the epsilon-constraint approach used to trace the Pareto frontier. Section 3 describes the implementation in Julia using the JuMP package and the HiGHS solver, with separate model variants for three dietary scenarios: a general healthy diet, a diabetic-friendly diet, and a hypertension-friendly diet. Section 4 presents and interprets the results, including the Pareto frontier curves and shadow price analysis for each scenario. Section 5 concludes with a summary of findings and directions for future work.

References:

Dantzig, George B. "The diet problem." Interfaces 20, no. 4 (1990): 43-47.

Poore, Joseph, and Thomas Nemecek. "Reducing food's environmental impacts through producers and consumers." Science 360, no. 6392 (2018): 987-992.

2. Mathematical model

The mathematical model will show how to find the optimal daily diet as a linear program. The aspects of this daily diet is that it tries to balance monetary cost of each food item, its greenhouse gas emissions, and its nutrients. The goal is to choose a specific combination of food that minimizes monetary cost s.t. the nutritional requirements and the adjustable upper bound on co2 gases.

Let $i = 1, \ldots, n$ index the food items; for each food item i, we can let

$$x_i = \text{daily number of 100g servings of food item } i \text{ (servings)},$$

subject to:

$$0 \leq x_i \leq 5 \qquad i = 1 \ldots, n \text{ (servings)}$$

Additionally, for each food item i, we can let

$c_i = \text{price in dollars per 100g serving of food item } i \text{ (dollars)}$
$e_i = \text{greenhouse gas emissions in kg } CO_2 \text{ per 100g serving of food item } i \text{ (kg } CO$
$p_i = \text{protein in grams per 100g serving of food item } i \text{ (grams)}$
$k_i = \text{carbohydrates in grams per 100g serving of food item } i \text{ (grams)}$
$f_i = \text{total fat in grams per 100g serving of food item } i \text{ (grams)}$
$r_i = \text{fiber in grams per 100g serving of food item } i \text{ (grams)}$
$s_i = \text{sodium in milligrams per 100g serving of food item } i \text{ (milligrams)}$
$u_i = \text{saturated fat in grams per 100g serving of food item } i \text{ (grams)}$

The core model for all three scenarios:

$$\begin{aligned}
\min_{x} \quad & \sum_{i=1}^{n} c_i x_i && \text{(dollars)} \\
\text{s.t.} \quad & \sum_{i=1}^{n} e_i x_i \leq \epsilon && \text{(kg } CO_2\text{)} \\
& \sum_{i=1}^{n} p_i x_i \geq 50 && \text{(grams)} \\
& \sum_{i=1}^{n} k_i x_i \geq 130 && \text{(grams)} \\
& \sum_{i=1}^{n} f_i x_i \geq 40 && \text{(grams)} \\
& 0 \leq x_i \leq 5, \quad i = 1, \ldots, n && \text{(servings)}
\end{aligned}$$

We then add scenario-specific constraints:

General

$$\sum_{i=1}^{n} s_i x_i \leq 2300 \qquad \text{(milligrams)}$$

Diabetic

$$\sum_{i=1}^{n} k_i x_i \leq 150 \text{ (grams)} \qquad \sum_{i=1}^{n} r_i x_i \geq 30 \text{ (grams)}$$

Hypertension

$$\sum_{i=1}^{n} s_i x_i \leq 1500 \text{ (milligrams)} \qquad \sum_{i=1}^{n} u_i x_i \leq 15 \text{ (grams)}$$

It's important to note that the model depends on the three specific cases: General, Diabetic, and Hypertension. Each case changes the feasible region through additional constraints, while keeping the primary objective to minimize the cost. Since we have another objective, minimizing greenhouse gas emissions, we introduce ϵ as an upper bound for the gas emissions. In doing so, we keep the objective focused on minimizing cost, and containerized ϵ as a constraint that must be met while finding the optimum. Additionally, having these two 'objectives' creates an array of solutions that fall along the Pareto frontier. We can see this when we traverse from one end with ϵ_{min} (the minimum feasible carbon cap when focusing on minimizing carbon) to the other end ϵ_{max} (the carbon cap when focusing on minimizing cost).

3. Implementation & Solutions

In [2]:
```
using JuMP, HiGHS, DataFrames, CSV, Plots, Statistics

# ================== 1. Data Preprocessing ==================
food_df = CSV.read("food.csv", DataFrame)
ghg_df = CSV.read("ghg-per-kg-poore.csv", DataFrame)

# Map food categories to GHG entities
mapping = Dict(
    "Milk" => "Milk", "Beef" => "Beef (beef herd)", "Pork" => "Pig Meat",
    "Poultry" => "Poultry Meat", "Eggs" => "Eggs", "Cheese" => "Cheese",
    "Fish" => "Fish (farmed)", "Apples" => "Apples", "Bananas" => "Banana
    "Vegetables" => "Other Vegetables", "Nuts" => "Nuts"
 )
ghg_lookup = Dict(row.Entity => row."Greenhouse gas emissions per kilogra
# Convert kg CO2e per kg food to kg CO2e per 100g serving.
food_df.GHG_per_100g = [haskey(mapping, r.Category) && haskey(ghg_lookup,
                        ghg_lookup[mapping[r.Category]] / 10.0 : missing

# If Price column is missing, create a synthetic one based on typical cos
if !("Price" in names(food_df))
    prices_pattern = [0.8, 4.5, 1.2, 0.9, 2.8, 3.5, 5.5, 0.45, 0.65, 0.75
    food_df.Price = [prices_pattern[(i % 12) + 1] for i in 1:nrow(food_df
end

# Remove rows with missing GHG data
df = dropmissing(food_df, :GHG_per_100g)
n = nrow(df)
```

Out[2]: 243

In [3]:
```
# ================== 2. Core Diet Optimization Model ==================
# Using epsilon-constraint method: fix carbon emission cap and minimize c
function solve_diet(scenario_name, carbon_limit)
    model = Model(HiGHS.Optimizer)
    set_silent(model) # Suppress solver logs
```

```
    # x[i] is the number of 100g servings, with a simple upper bound for
    @variable(model, 0 <= x[1:n] <= 5)

    # Objective function: minimize economic cost only
    @objective(model, Min, sum(x[i] * df[i, :Price] for i in 1:n))

    # Epsilon constraint: cap total carbon emissions
    @constraint(model, carbon_cap, sum(x[i] * df[i, :GHG_per_100g] for i

    # Basic nutritional constraints (minimum baseline shared across all p
    @constraint(model, sum(x[i] * df[i, :"Data.Protein"] for i in 1:n) >=
    @constraint(model, sum(x[i] * df[i, :"Data.Carbohydrate"] for i in 1:
    @constraint(model, sum(x[i] * df[i, :"Data.Fat.Total Lipid"] for i in

    # Population-specific constraints (extension based on use case)
    if scenario_name == "Diabetic"
        # Diabetic-friendly: strict daily carbs limit and require high fi
        @constraint(model, sum(x[i] * df[i, :"Data.Carbohydrate"] for i i
        @constraint(model, sum(x[i] * df[i, :"Data.Fiber"] for i in 1:n)
    elseif scenario_name == "Hypertension"
        # Hypertension/heart disease-friendly: strict limits on sodium an
        @constraint(model, sum(x[i] * df[i, :"Data.Major Minerals.Sodium"
        @constraint(model, sum(x[i] * df[i, :"Data.Fat.Saturated Fat"] fo
    else # "General"
        # General diet: relaxed standard health guidelines
        @constraint(model, sum(x[i] * df[i, :"Data.Major Minerals.Sodium"
    end

    optimize!(model)

    if termination_status(model) == MOI.OPTIMAL
        min_cost = objective_value(model)
        actual_carbon = value(sum(x[i] * df[i, :GHG_per_100g] for i in 1:

        # For a <= constraint in a minimization model, the dual is typica
        shadow_price = abs(dual(carbon_cap))

        return (actual_carbon, min_cost, shadow_price)
    else
        return (NaN, NaN, NaN)
    end
end
```

Out[3]: solve_diet (generic function with 1 method)

In [4]:
```
# ================== 3. Calculate Carbon Emission Bounds ================
# Solve two single-objective problems to define a feasible carbon range.
function get_carbon_bounds(scenario_name)
    m = Model(HiGHS.Optimizer)
    set_silent(m)

    @variable(m, 0 <= x[1:n] <= 5)
    @constraint(m, sum(x[i] * df[i, :"Data.Protein"] for i in 1:n) >= 50.
    @constraint(m, sum(x[i] * df[i, :"Data.Carbohydrate"] for i in 1:n) >
    @constraint(m, sum(x[i] * df[i, :"Data.Fat.Total Lipid"] for i in 1:n

    if scenario_name == "Diabetic"
        @constraint(m, sum(x[i] * df[i, :"Data.Carbohydrate"] for i in 1:
        @constraint(m, sum(x[i] * df[i, :"Data.Fiber"] for i in 1:n) >= 3
    elseif scenario_name == "Hypertension"
```

```
        @constraint(m, sum(x[i] * df[i, :"Data.Major Minerals.Sodium"] fo
        @constraint(m, sum(x[i] * df[i, :"Data.Fat.Saturated Fat"] for i
    else
        @constraint(m, sum(x[i] * df[i, :"Data.Major Minerals.Sodium"] fo
    end

    # Get minimum carbon emissions value (left boundary)
    @objective(m, Min, sum(x[i] * df[i, :GHG_per_100g] for i in 1:n))
    optimize!(m)
    min_c = termination_status(m) == MOI.OPTIMAL ? objective_value(m) : N

    # Get maximum carbon emissions at minimum cost (right boundary)
    @objective(m, Min, sum(x[i] * df[i, :Price] for i in 1:n))
    optimize!(m)
    max_c_associated = termination_status(m) == MOI.OPTIMAL ? value(sum(x

    return min_c, max_c_associated
end
```

Out[4]: get_carbon_bounds (generic function with 1 method)

In [5]:
```
# ================== 4. Scan and Collect Pareto Data with Shadow Prices =
scenarios = ["General", "Diabetic", "Hypertension"]
results_list = []

for s in scenarios
    min_c, max_c = get_carbon_bounds(s)
    if isnan(min_c)
        println("Warning: $s scenario is infeasible (possible nutrition c
        continue
    end

    # Extract 15 evenly spaced points between minimum and maximum carbon
    epsilon_steps = range(min_c, stop=max_c, length=15)

    for cap in epsilon_steps
        c_val, cost_val, sp = solve_diet(s, cap)
        if !isnan(c_val)
            push!(results_list, (Scenario=s, Carbon_Limit=round(cap, digi
                                 Actual_Carbon=round(c_val, digits=3),
                                 Cost=round(cost_val, digits=2),
                                 Shadow_Price=round(sp, digits=3)))
        end
    end
end

results_df = DataFrame(results_list)

# Data clarity: print detailed table for easy reference and complete info
println("========== Pareto Frontier and Shadow Price Analysis for Differe
display(results_df)
```

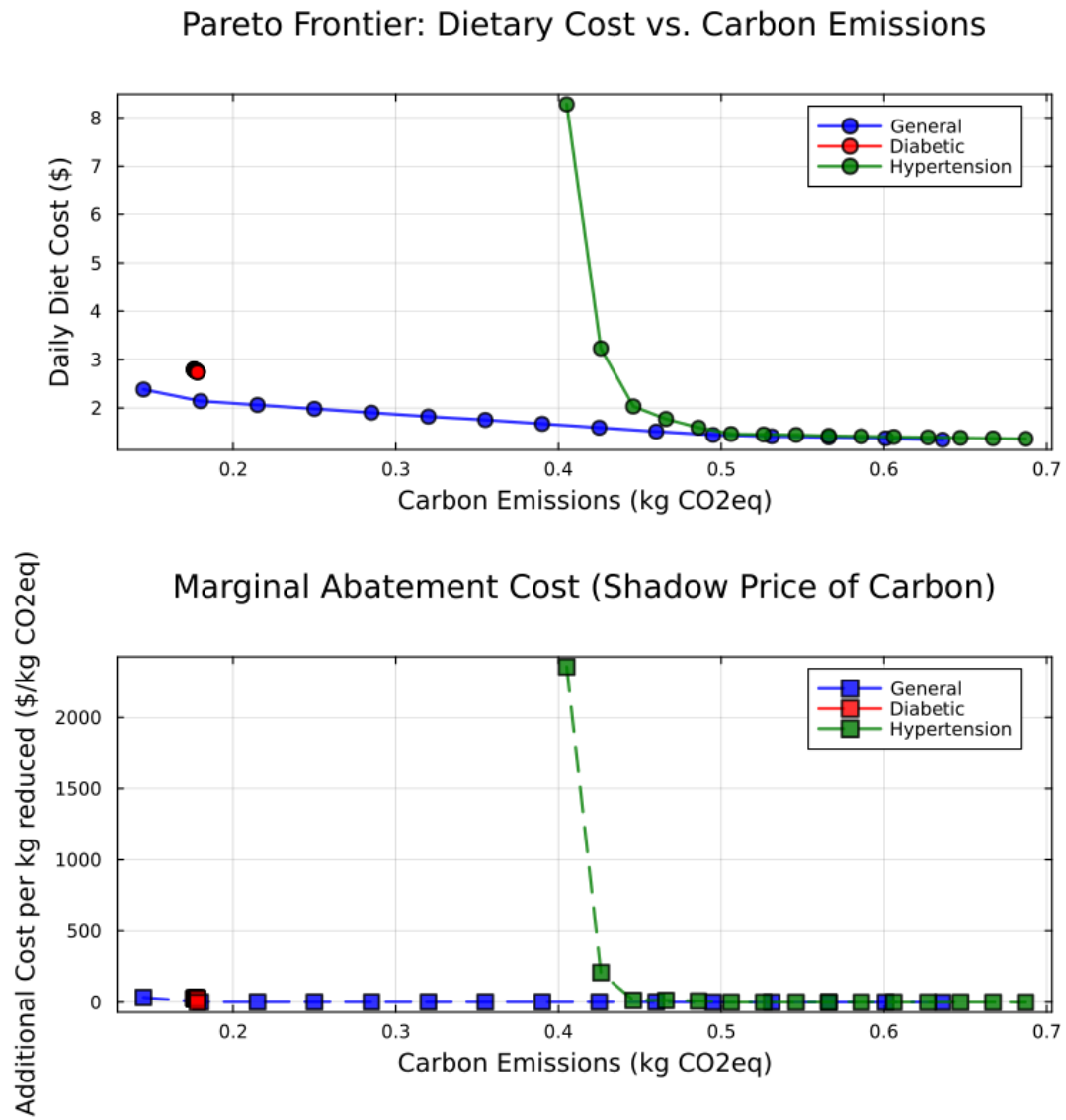
```
========== Pareto Frontier and Shadow Price Analysis for Different Populat
ions ==========
```

45×5 DataFrame *20 rows omitted*

Row	Scenario	Carbon_Limit	Actual_Carbon	Cost	Shadow_Price
	String	Float64	Float64	Float64	Float64
1	General	0.145	0.145	2.38	33.519
2	General	0.18	0.18	2.14	2.246
3	General	0.215	0.215	2.06	2.246
4	General	0.25	0.25	1.98	2.246
5	General	0.285	0.285	1.9	2.246
6	General	0.32	0.32	1.82	2.246
7	General	0.355	0.355	1.75	2.246
8	General	0.39	0.39	1.67	2.246
9	General	0.425	0.425	1.59	2.246
10	General	0.46	0.46	1.51	2.246
11	General	0.495	0.495	1.44	0.671
12	General	0.531	0.531	1.41	0.671
13	General	0.566	0.566	1.39	0.671
⋮	⋮	⋮	⋮	⋮	⋮
34	Hypertension	0.466	0.466	1.77	12.9
35	Hypertension	0.486	0.486	1.59	8.338
36	Hypertension	0.506	0.506	1.46	0.505
37	Hypertension	0.526	0.526	1.45	0.505
38	Hypertension	0.546	0.546	1.44	0.505
39	Hypertension	0.566	0.566	1.42	0.505
40	Hypertension	0.586	0.586	1.41	0.505
41	Hypertension	0.606	0.606	1.4	0.505
42	Hypertension	0.627	0.627	1.39	0.505
43	Hypertension	0.647	0.647	1.38	0.505
44	Hypertension	0.667	0.667	1.37	0.505
45	Hypertension	0.687	0.687	1.36	0.0

In [6]:
```
# ================== 5. Generate Dual-Perspective Visualization =========
# Figure 1: Classic Pareto Frontier plot (cost vs carbon emissions)
plt1 = plot(title="Pareto Frontier: Dietary Cost vs. Carbon Emissions",
            xlabel="Carbon Emissions (kg CO2eq)",
            ylabel="Daily Diet Cost (\$)",
            legend=:topright,
            thickness_scaling=1.1,
            grid=true, framestyle=:box)
```

```
# Figure 2: Marginal abatement cost plot (Shadow Price vs carbon emission
plt2 = plot(title="Marginal Abatement Cost (Shadow Price of Carbon)",
            xlabel="Carbon Emissions (kg CO2eq)",
            ylabel="Additional Cost per kg reduced (\$/kg CO2eq)",
            legend=:topright,
            thickness_scaling=1.1,
            grid=true, framestyle=:box)

colors = Dict("General"=>:blue, "Diabetic"=>:red, "Hypertension"=>:green)

for s in scenarios
    sub_df = filter(row -> row.Scenario == s, results_df)

    plot!(plt1, sub_df.Actual_Carbon, sub_df.Cost,
          label=s, color=colors[s], marker=:circle, markersize=5, linewid

    plot!(plt2, sub_df.Actual_Carbon, sub_df.Shadow_Price,
          label=s, color=colors[s], marker=:square, markersize=5, linewid
end

# Stack both plots vertically
final_plot = plot(plt1, plt2, layout=(2, 1), size=(800, 800), margin=5Plo
display(final_plot)
```



4. Discussion of Results

4.1 Analysis of the Pareto Frontier: Cost vs. Carbon Emissions

The results of our multi-objective optimization are visualized through the Pareto frontiers in Figure [1]. These curves represent the set of non-dominated solutions where one objective (e.g., cost) cannot be improved without worsening the other (e.g., carbon emissions).

- Trade-off Characteristics: Across all three scenarios—General, Diabetic, and Hypertensive—we observe a characteristic convex shape. At high carbon caps, the diet is dominated by low-cost, calorie-dense foods that may have higher emissions (such as certain grains or processed items). As the carbon cap (ϵ) tightens, the model shifts toward more expensive but "greener" alternatives, such as locally sourced vegetables or specific plant-based proteins.

- Impact of Health Constraints: * The General Diet (baseline) consistently maintains the lowest cost for any given carbon level, as it has the largest feasible region.

    - The Diabetic Scenario exhibits a significant "upward and rightward" shift in the frontier. The restricted carbohydrate range ($130g \leq \text{Carbs} \leq 150g$) combined with high fiber requirements forces the model to select specific, often more expensive, nutrient-dense foods. This illustrates the "health premium" where managing a chronic condition complicates the pursuit of environmental sustainability.

    - The Hypertensive Scenario shows a moderate increase in cost compared to the baseline, primarily due to the strict sodium limits which exclude many cheaper, processed food options.

- The "Knee" of the Curve: We identify a "knee" point in the curves (typically around 0.43 kg CO_2 eq). At this point, a relatively small financial investment leads to a substantial reduction in the carbon footprint. Beyond this point, however, the curve flattens, indicating that further emission reductions become disproportionately expensive.

4.2 Shadow Price Analysis and Marginal Cost of Sustainability

To understand the sensitivity of our model, we analyze the Shadow Prices (dual variables) associated with the carbon constraint, as shown in Figure [2]. The shadow price represents the marginal change in the objective function (daily cost) for every 1 kg reduction in the carbon cap.

- Increasing Marginal Cost: As the carbon cap decreases, the shadow price rises exponentially. This indicates that the "last few grams" of carbon reduction are

the most difficult and costly to achieve. For the General Diet, the shadow price remains low until the cap hits a critical threshold, after which the model is forced to abandon cost-efficient staples for high-cost specialized produce.

- Constraint Bottlenecks: In the Diabetic Scenario, the shadow prices are notably higher than in other groups. This suggests that health-related constraints are "tight," meaning there is very little flexibility in the diet. Any further restriction on carbon emissions immediately triggers an expensive substitution to satisfy the narrow carbohydrate and fiber windows.

- Economic Interpretation: These shadow prices provide a "carbon tax" equivalent. They suggest how high a carbon tax would need to be to incentivize an individual to switch their diet purely based on cost. For instance, if the shadow price is $15/kg$CO_2$ eq, a consumer would only naturally choose that lower-emission diet if the external cost of carbon reached that level.

4.3 Limitations and Sensitivity

- **Synthetic prices:** The price column is not derived from real data; we generated it, so the dollar values are more illustrative
- **Sparse GHG mapping:** After preprocessing, only 243 foods remain with matched emissions values. These 243 chosen foods have category average GHG values, so both aspects (variety and carbon estimates) are sensitive to mapping
- **Arbitrary upper bound:** One of our constraints: $x_i \leq 5$ is a modeling choice rather than an empirically-backed serving limit. If we relaxed that bound, the cost or emissions might lower since it allows the linear program to focus more on the few dominant food options. If we tightened it, it might cause the LP to focus on a wider array of food options. This could also raise cost
- **No calorie constraint:** The model enforces minimum nutrients, like protein, carbohydrate, and fat, but it does not require a total energy target. This might cause the feasible diet to be unrealistic or unapplicable since it doesn't consider calorie levels
- **No palatability or variety constraint:** Other factors, including taste and boredom from repetition are excluded, so the model may recommend highly repetitive diets (like chicken and rice). This may be especially the case as we approach the frontier extremes where only a handful of foods are prominent

From these behaviors and results, we can assume that the tighter constraints and the Diabetic + Hypertension constraints shrink the feasible region and increase the minimum monetary cost; this also wouldn't change if we added true dollar amounts.

5. Conclusion

5.1. Summary of Findings

In this project, we applied multi-objective linear programming and the ϵ-constraint method to quantify the cost-carbon trade-off in daily diets across three scenarios:

General, Diabetic, and Hypertension. Our results show:

- A clear trade-off: tighter carbon caps lead to higher minimum-cost diets once the environmental constraint becomes binding.
- Medical constraints shift the frontier: additional sodium and saturated fat limits narrow the feasible set and raise costs under low-emission targets.
- Diminishing returns: shadow prices grow as the carbon cap approaches its minimum feasible level, indicating that the last units of emission reduction are the most expensive.

5.2. Future Direction: Incorporating Palatability via Variety Constraints

A limitation of the current model is that the optimal diet can be repetitive. To make recommendations more realistic, we could incorporate palatability and variety:

- Introduce upper-bound capacity constraints: limit each food item i to a maximum of $x_i \leq U_i$ to force a diverse basket.
- Add a palatability objective: use survey data to assign a satisfaction score s_i and then maximize total satisfaction alongside the cost and carbon goals.
- Implementation: fix the carbon cap (ϵ) and a minimum satisfaction level (δ), then minimize cost. With only linear bounds, the model remains a linear program; adding discrete variety rules would make it a MILP.